Probabilistic Forecasting via Autoregressive Flow Matching

Abstract

In this work, we introduce autoregressive flow matching (AFM) for probabilistic forecasting of multivariate timeseries data. Given historical measurements and optional future covariates, we formulate forecasting as sampling from a learned conditional distribution over future trajectories. We decompose the joint distribution of future observations into a sequence of conditional densities, each modeled via a shared flow that transforms a simple base distribution into the next observation distribution, conditioned on observed covariates. We leverage the flow matching framework, enabling scalable and simulation-free learning of these transformations. By combining this factorization with the flow matching objective, AFM retains the benefits of classical autoregressive models, including strong extrapolation performance, compact model size and improved marginal calibration relative to whole-window flow matching approaches, while also capturing complex multi-modal conditional distributions, as seen in modern transport-based generative models. We demonstrate the effectiveness of AFM on multiple stochastic dynamical systems and real-world forecasting tasks.

Keywords: probabilistic forecasting, flow matching, autoregressive models, generative models, multivariate time series

1. Introduction

A core problem in statistics, econometrics, and machine learning is probabilistic timeseries forecasting, where the aim is to extrapolate how system dynamics evolve into the future given observational data. This problem is central to a wide range of scientific, industrial and societal disciplines (Lim and Zohren, 2021; Dama and Sinoquet, 2021; Ye et al., 2024). Classical approaches based on ARIMA (Box et al., 1994) and exponential smoothing (Hyndman et al., 2008) have long provided tractable probabilistic fore-

casts for univariate and low-dimensional settings, and remain competitive benchmarks as demonstrated in the M-competition series (Makridakis et al., 2020). However, extending these methods to high-dimensional multivariate settings with complex, potentially multimodal predictive distributions motivates the use of flexible deep generative models.

An emerging trend is to leverage deep generative models to tackle this problem (Karl et al., 2016; Rasul et al., 2021b; Desai et al., 2021). In this setting, forecasting is framed as sampling from a future probability density conditioned on the past. Most notably, diffusion models and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021) have recently pushed state-of-the-art performance in multiple forecasting benchmarks (Rasul et al., 2021a; Tashiro et al., 2021; Kollovieh et al., 2023; Meijer and Chen, 2024). Despite their strong empirical performance, diffusion models incur substantial computational costs at both training and inference time.

Flow matching (FM) (Liu et al., 2023; Albergo et al., 2023; Lipman et al., 2023) is an emerging paradigm for generative modeling that generalizes and subsumes diffusion models while offering more flexible design choices. Unlike diffusion models, which rely on iterative stochastic denoising steps over a discretized trajectory, FM learns deterministic probability paths that transforms arbitrary base distributions into the target distribution directly through continuous normalizing flows. By optimally designing these probability paths, FM circumvents the need for handcrafted noise schedules and lengthy ancestral sampling chains inherent to diffusion, enabling more efficient training and sampling.

Recently, FM has been applied in the context of timeseries modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kollovieh et al., 2025; Liu et al., 2026), showing strong empirical results and improved computational efficiency compared to diffusion models. However, current approaches rely on directly learning the conditional distribution of an entire fixed-horizon future window conditioned on a fixed-horizon context window. While this leads to fast training and sampling, this design choice may limit extrapolation performance beyond the training horizon and can result in less well-calibrated marginal uncertainty estimates compared to step-wise autoregressive approaches. Additionally, it introduces a more complex optimization problem, where models attempt to learn intricate time-dependent probability paths along the flow dimension. The latter issue is highlighted by Kollovieh et al. (2025), who propose to use informed priors based on Gaussian pro-

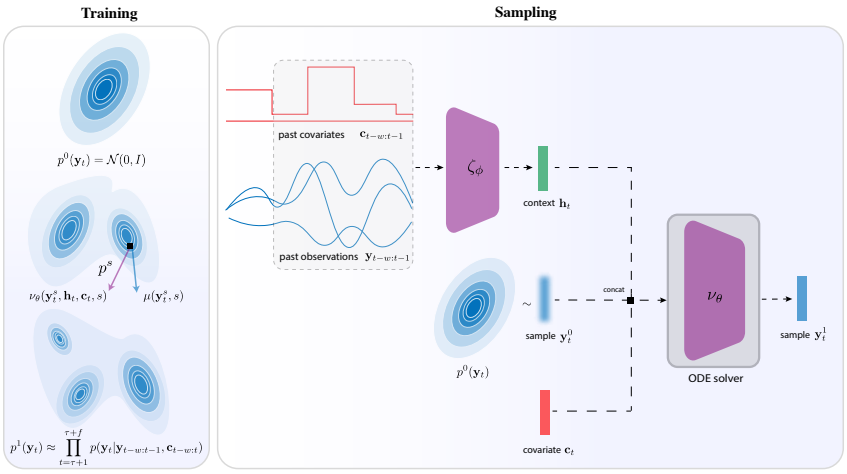


Figure 1: Overview of the AFM training and inference pipeline. During training, a probability path $(p^s)^{0 \leq s \leq 1}$ is constructed between a base distribution p^0 and the target distribution p^1. This probability path is generated by a velocity field μ. Training is done by regressing μ via a neural network ν_θ which takes in a sample from the probability path $\mathbf{y}_t^s$ at flow step s, a context vector $\mathbf{h}_t$ encoding past observations and covariates, and the current covariate $\mathbf{c}_t$. After training, sampling from the target distribution is achieved via first sampling the base distribution p^0, and integrating the trained velocity field via a ODE solver until $s = 1$.

cesses to simplify and speed up optimization. However, this restricts their approach to univariate forecasting problems.

In this work, we propose autoregressive flow matching (AFM) as a simple alternative that utilizes FM to learn probabilistic forecasting models that scale to high-dimensional multivariate datasets without relying on informed priors. Unlike existing FM-based methods such as TSFlow (Kollovieh et al., 2025) that model the entire future window simultaneously (and are consequently restricted to univariate settings due to their GP-based priors), AFM decomposes the forecasting problem into a sequence of conditional distributions, modeling the distribution of each future time step conditioned on past observations and covariates. This is achieved using a shared flow that transforms a simple base distribution into the next observation's distribution. This autoregressive structure, shown in Fig. 1, enables AFM to naturally handle variable forecasting horizons, extrapolate beyond the training distribution, and provide improved marginal calibration relative to whole-window flow matching approaches, while simplifying the optimization problem.

We demonstrate that AFM achieves superior or competitive performance on both classical dynamical systems and real-world forecasting tasks, offering a practical and effective solution to probabilistic timeseries forecasting.

2. Related Work

Classical and statistical forecasting. The forecasting literature has a long tradition of methods grounded in statistics and econometrics. The Box–Jenkins ARIMA family (Box et al., 1994) and exponential smoothing state space models (Hyndman et al., 2008) remain strong benchmarks for point and interval forecasting, as repeatedly demonstrated in the M-competition series (Makridakis et al., 2020). Probabilistic extensions of these classical methods produce prediction intervals under Gaussian or parametric assumptions, with evaluation via proper scoring rules such as the continuous ranked probability score (Gneiting and Raftery, 2007) and the Winkler score (Winkler et al., 1996). A comprehensive survey spanning statistical and machine learning forecasting approaches is provided by Petropoulos et al. (2022). AFM adopts the CRPS as its primary evaluation metric, following established practice in this literature (Hyndman and Koehler, 2006).

Deep learning for probabilistic forecasting. Recurrent neural networks and their variants have been widely applied to time series forecasting. DeepAR (Salinas et al., 2020) introduced autoregressive recurrent networks that learn flexible parametric output distributions (Gaussian, negative binomial), achieving competitive performance across a wide range of benchmarks. WaveNet (Van Den Oord et al., 2016) adapted dilated causal convolutions to sequence generation, and attention-based architectures have been explored for capturing long-range dependencies (Lim and Zohren, 2021). These approaches typically assume a fixed parametric output distribution, which may be insufficient for complex multimodal or heavy-tailed predictive distributions.

Deep generative models for forecasting. To move beyond parametric assumptions, generative models have been applied to learn flexible predictive distributions. Normalizing flows (Rasul et al., 2021b) and variational autoencoders (Desai et al., 2021; Karl et al., 2016) provide tractable density estimation for multivariate time series. Diffusion models (Ho et al., 2020; Song et al., 2021) have achieved strong empirical results: TimeGrad (Rasul et al., 2021a) applies autoregressive denoising diffusion to multivariate forecasting;

CSDI (Tashiro et al., 2021) uses conditional score-based models for imputation and forecasting; SSSD (Alcaraz and Strodthoff, 2023) combines diffusion with structured state spaces; and TSDiff (Kollovieh et al., 2023) proposes self-guided diffusion. Despite their strong performance, these models incur substantial computational costs at both training and inference time.

Flow matching for time series. Flow matching (Lipman et al., 2023; Liu et al., 2023; Albergo et al., 2023) generalizes diffusion by learning deterministic probability paths via continuous normalizing flows, enabling more flexible trajectory design and efficient training. Recent work has applied FM to time series: TSFlow (Kollovieh et al., 2025) uses Gaussian process priors for probabilistic forecasting but is restricted to univariate settings; FM-TS (Hu et al., 2024) and CoGenCast (Liu et al., 2026) are concurrent FM-based approaches for time series generation and forecasting; and Tamir et al. (2024) study conditional FM for time series modelling. The present work differs from these by introducing an autoregressive factorization within the FM framework, enabling multivariate forecasting without informed priors and improving marginal calibration relative to whole-window FM approaches.

3. Autoregressive flow matching

3.1. Flow matching

Standard flow matching (Lipman et al., 2023) is a paradigm for generative modeling that enables training of continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2019) at an unprecedented scale. CNFs model data transformations as solutions to ordinary differential equations (ODEs), providing invertible mappings with tractable likelihoods.

Similar to CNFs, FM aims to learn a time-dependent diffeomorphic map defined on the data space Ω, called a flow $\psi\colon [0,1] \times \Omega \to \Omega$. This flow transforms a sample $\mathcal{X}^0 \sim p$ from a source distribution p into a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ such that $\mathcal{X}^1 \sim q$ for some target distribution q. The flow is constructed by solving the following initial value problem:

$$\mathrm{d}\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)\mathrm{d}s \text{ s. t. } \psi(\mathcal{X}, 0) = \mathcal{X} \tag{1}$$

where $\mu\colon \Omega \times [0,1] \to \Omega$ is a vector field defining the velocity of the flow and generating a probability path $(p^s)^{0 \leq s \leq 1}$ where each p^s is a distribution over Ω with $p^0 = p$. The objective is thus to learn a valid vector field μ such that $p^1 = q$.

Unlike CNFs where the vector field is learned via likelihood maximization, requiring solving and differentiating through Eq. (1) during training, FM learns a vector field ν_θ with parameters θ by regressing over vector fields of fixed conditional probability paths. This enables defining a tractable objective function called conditional flow matching (CFM) defined as:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}\left\|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s \mid \mathcal{X}^1)\right\|^2 \tag{2}$$

where the expectation is taken over $s \sim \mathcal{U}[0,1]$, $\mathcal{X}^1 \sim q(\mathcal{X}^1)$ and $\mathcal{X} \sim p^s(\mathcal{X} \mid \mathcal{X}^1)$.

Minimizing this objective allows sampling from q by solving Eq. (1) with the trained ν_θ. By avoiding gradient backpropagation through the ODE solver, FM enables stable and scalable training for high-dimensional generative tasks, such as image and video modeling (Esser et al., 2024; Polyak et al., 2024). Various probability path designs and conditioning strategies have been proposed, unifying multiple transport-based generative models under this framework (Liu et al., 2023; Albergo et al., 2023; Tong et al., 2024).

3.2. Autoregressive flow matching

We propose an autoregressive extension of the flow matching paradigm with the aim of improving probabilistic forecasting.

Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n-dimensional observation at time τ. Given access to its l past measurements including the current observation, denoted as $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \ldots, \mathbf{y}_\tau\}$, our objective is to provide probabilistic forecasts of the next f future values denoted as $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \ldots, \mathbf{y}_{\tau+f}\}$, potentially conditioned on observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \ldots, \mathbf{c}_{\tau+f}\}$ where $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, our goal is to learn to sample from the conditional distribution $p(\mathbf{Y}_f \mid \mathbf{Y}_l, \mathbf{C})$ using a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$ with m the number of instances.

We propose to factorize the conditional distribution autoregressively across future time-steps under a Markov assumption of order w such that:

$$p(\mathbf{Y}_f \mid \mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t \mid \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \tag{3}$$

where l is the total context available at inference time and $w \leq l$ is the Markov order, i.e., the number of past steps each conditional depends on.

During training, the context window $\{\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}\}$ is filled with ground-truth observations (teacher forcing); during inference it is filled with the model's own rolling predictions. Our objective thus becomes learning the conditional distribution in Eq. (3). Instead of maximum likelihood estimation, as commonly used for autoregressive models, we employ the flow matching framework to learn this conditional distribution.

This autoregressive factorization provides several advantages. First, it reduces the learning problem to modeling simpler conditional distributions $p(\mathbf{y}_t \mid \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ rather than a complex high-dimensional joint distribution. Second, each time step's conditional distribution can be trained within a teacher forcing framework allowing parallel training. Third, the Markov assumption with window size w offers a trade-off between model capacity and computational complexity. Finally, this formulation enables interpretable uncertainty quantification by explicitly modeling how prediction uncertainty propagates through time via the chain of conditional distributions.

In autoregressive flow matching, for all future time steps $t \in [\tau+1, \tau+f]$, we define a target probability path $(p^s(\mathbf{y}_t))_{0\leq s\leq 1}$, with marginals $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and

$$p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t \mid \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$$

where $p_{\mathcal{D}}$ is the empirical distribution. This probability path is generated by a flow field $\psi\colon \mathbb{R}^n \times [0,1] \to \mathbb{R}^n$ along s defined via the ordinary differential equation (ODE):

$$\mathrm{d}\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)\mathrm{d}s \text{ s. t. } \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \tag{4}$$

where $\mu\colon \mathbb{R}^n \times [0,1] \to \mathbb{R}^n$ is a vector field constructing the flow chosen such that the resulting flow satisfy the continuity equation.

To derive a tractable objective, we can model the marginal probability path as a mixture of conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t \mid \mathbf{z}) p(\mathbf{z}) \mathrm{d}\mathbf{z} \tag{5}$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning random variable sampled from an arbitrary data coupling $\pi_{0,1}$. Theorem 2 in Lipman et al. (2023) shows that the conditional velocity fields $\mu(\mathbf{y}_t, s \mid \mathbf{z})$ associated with the conditional probability paths $p^s(\mathbf{y}_t \mid \mathbf{z})$ are equivalent to their respective marginal velocity field $\mu(\mathbf{y}_t, s)$.

Following Tong et al. (2024), we choose $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are samples from p^0 and p^1, respectively. We define the conditional probability path as:

$$p^s(\mathbf{y}_t \mid \mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \tag{6}$$

with marginals satisfying $p^0(\mathbf{y}_t \mid \mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t \mid \mathbf{z}) = p^1(\mathbf{y}_t)$ for $\sigma^2 \to 0$. The respective conditional velocity field is then simply given by $\mu(\mathbf{y}_t, s \mid \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This choice offers straight probability paths from the source to the target distributions in Euclidean space and closed-form solutions, enabling efficient training and sampling. Learning this velocity field allows us to sample from our target distribution $p(\mathbf{y}_t \mid \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ by solving the ODE in Eq. (4) until $s = 1$.

3.3. AFM training

Similar to the standard FM setting, we learn the velocity field by regressing it against a neural network ν_θ with parameters θ using the conditional flow matching objective in Eq. (2). To efficiently achieve this in our setting, we define a context vector $\mathbf{h}_t \in \mathbb{R}^h$ that encodes the Markov context window at time t defined as:

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \tag{7}$$

where $\zeta \colon \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \to \mathbb{R}^h$ is a neural network with parameters ϕ, and w is the Markov order from Eq. (3). Note that $l \geq w$: the full context $\mathbf{Y}_l$ is available at the start of inference to initialise the rolling window, but each conditional only conditions on the most recent w steps. We can then define our learning objective as:

$$\mathcal{L}(\theta, \phi) = \mathbb{E} \left\| \mu(\mathbf{y}_t^s, s \mid \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s) \right\|^2 \tag{8}$$

where the expectation is taken over $\mathbf{z} \sim \pi_{0,1}$, $s \sim \mathcal{U}(0,1)$ and $\mathbf{y}_t^s \sim p^s(\mathbf{y}_t \mid \mathbf{z})$. Training proceeds by randomly sampling an observation at time point t, its associated context window and covariates from $\mathcal{D}$, and jointly optimising the parameters ϕ and θ by minimizing $\mathcal{L}(\theta, \phi)$ via stochastic gradient descent. Note that this is akin to teacher forcing (Williams and Zipser, 1989), where ground-truth past observations are fed into the model during training (rather than its own past predictions), allowing parallel training across all time-steps and avoiding error accumulation. Algorithm 1 summarizes the training procedure for autoregressive flow matching.

Algorithm 1 AFM training.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w, networks ν_θ, ζ_ϕ

while not converged **do**
  Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
  Sample $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
  Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
  Sample $s \sim \mathcal{U}(0, 1)$
  Sample $\mathbf{y}_t^s \sim \mathcal{N}\big((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I\big)$
  Set target $\mu(\mathbf{y}_t^s, s \mid \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
  Set $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s \mid \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
  Update θ, ϕ to minimize the loss
end while
return ν_θ, ζ_ϕ

3.4. AFM inference

After training, we wish to provide probabilistic predictions for new observations $\mathbf{Y}_l$ and corresponding covariates $\mathbf{C}$ for f future timestamps into the future. For this we follow the sampling procedure in Algorithm 2, where predictions are made autoregressively using a rolling-window approach. This procedure can be repeated many times to obtain empirical quantiles of the uncertainty of our predictions.

Algorithm 2 AFM inference.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f, window size w, networks ν_θ, ζ_ϕ

Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
for $t = \tau + 1$ **to** $\tau + f$ **do**
  Set context $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$ using the last w entries in $\mathbf{Y}_f$
  Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
  Set $\psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0$
  Solve $\frac{\mathrm{d}\psi(\mathbf{y}_t, s)}{\mathrm{d}s} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s)$ for $s \in [0, 1]$
  Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
end for
return $\mathbf{Y}_f$

Table 1: Summary of stochastic dynamical systems. $\mathbf{f}(\mathbf{x})$ denotes the drift function, $\boldsymbol{\Theta}$ the system parameters, $\boldsymbol{\sigma}$ the constant diffusion term, and $p(\mathbf{x}(0))$ the initial condition distribution. Each system was integrated over the given interval $[t_0, t_1]$ with 200 steps using the Euler–Heun method.

System	$f(\mathbf{x})$	$\boldsymbol{\Theta}$	$\boldsymbol{\Sigma}$	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma\,(x_2 - x_1)$ $f_2 = x_1(\rho - x_3) - x_2$ $f_3 = x_1\,x_2 - \beta\,x_3$	$\sigma = 10$ $\rho = 28$ $\beta = \frac{8}{3}$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	$[0, 2]$
FitzHugh–Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$ $f_2 = \frac{x_1 + a - b\,x_2}{\tau}$	$a = 0.7$ $b = 0.8$ $\tau = 12.5$ $I = 0.5$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	$[0, 10]$
Lotka–Volterra	$f_1 = \alpha\,x_1 - \beta\,x_1\,x_2$ $f_2 = -\delta\,x_2 + \gamma\,x_1\,x_2$	$\alpha = 1.3$ $\beta = 0.9$ $\gamma = 0.8$ $\delta = 1.8$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	$[0, 20]$
Brusselator	$f_1 = A + x_1^2\,x_2 - (B + 1)\,x_1$ $f_2 = B\,x_1 - x_1^2\,x_2$	$A = 1.0$ $B = 3.0$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	$[0, 20]$
Van der Pol	$f_1 = x_2$ $f_2 = \mu\,(1 - x_1^2)\,x_2 - x_1$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	$[0, 20]$

4. Experimental validation

In this section, we present our empirical results and compare AFM against various baselines using both data generated via simulating various dynamical systems and publicly available data for real-world use cases.

4.1. Simulation experiments

Data. We evaluated AFM on the task of forecasting five different stochastic dynamical systems: stochastic versions of the Lorenz system (Lorenz, 1963), FitzHugh–Nagumo model (FitzHugh, 1961), Lotka-Volterra system (Lotka, 1925), Brusselator (Lefever and Prigogine, 1968), and Van der Pol oscillator (van der Pol, 1926).

For each system, we generated trajectories by numerically solving the corresponding stochastic differential equations (SDEs), using randomly sampled initial conditions and Brownian motion realizations. Each system has the general form

$$\mathrm{d}\mathbf{x}(t) \;=\; \mathbf{f}\big(\mathbf{x}(t), t; \boldsymbol{\Theta}\big)\,\mathrm{d}t \;+\; \boldsymbol{\sigma}\,\mathrm{d}\mathbf{W}(t)$$

where f is the deterministic drift, $\boldsymbol{\Theta}$ are the system parameters, $\boldsymbol{\sigma}$ is a constant diffusion term, and $\mathbf{W}(t)$ is standard Brownian motion. We sampled 2400 random initial conditions from a uniform distribution over the specified range (see Table 1) for each system and generated a trajectory for each initial condition by solving the SDE using the Euler-Heun method with fixed step size. We sampled 200 equally spaced time points for each trajectory. Out of the 200 time points in each trajectory, the first 75 were used as observed data, the next 75 for prediction, and the final 50 for extrapolation. Out of the 2400 trajectories generated, we used 2000 for training and 400 for testing.

Implementation details. For these experiments we set ζ_ϕ as a two-layer bi-directional LSTM with 64 hidden units, and set ν_θ as a multi-layer perceptron with three hidden layers of size 64. We use Fourier positional embeddings to encode the flow step s into a 16-dimensional vector using a fixed set of frequencies and set the context length w to be equal to the prediction length. We trained our models via stochastic gradient descent using an Adam optimizer with a learning rate of 0.003 and a batch size of 128. Training until convergence took $\sim$7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Table 2: Performance comparison of autoregressive (AFM) vs non-autoregressive (FM) factorization in our method for five different stochastic dynamical systems. Performance is compared in terms of average CRPS and NRMSE on the 400 held-out test trajectories, no part of which is seen during training. Scores are reported separately for forecast steps within the horizon used during training (steps 76–150) and beyond it (steps 151–200). Reported results are the mean and standard deviation across five different random seeds. Best results in bold.

System	**Factorization**	**Within training horizon** (steps 76–150)		**Beyond training horizon** (steps 151–200)	
		$\overline{\text{CRPS}}\downarrow$	NRMSE $\downarrow$	$\overline{\text{CRPS}}\downarrow$	NRMSE $\downarrow$
Lorenz	FM	0.147 ± 0.012	0.242 ± 0.018	$\mathbf{0.120} \pm 0.009$	0.301 ± 0.024
	AFM (ours)	$\mathbf{0.090} \pm 0.008$	$\mathbf{0.017} \pm 0.003$	0.137 ± 0.011	$\mathbf{0.048} \pm 0.006$
FitzHugh-Nagumo	FM	0.165 ± 0.014	0.131 ± 0.011	0.214 ± 0.016	0.306 ± 0.022
	AFM (ours)	$\mathbf{0.144} \pm 0.013$	$\mathbf{0.090} \pm 0.007$	$\mathbf{0.146} \pm 0.012$	$\mathbf{0.278} \pm 0.019$
Lotka-Volterra	FM	0.130 ± 0.011	0.266 ± 0.019	0.136 ± 0.012	0.502 ± 0.033
	AFM (ours)	$\mathbf{0.126} \pm 0.010$	$\mathbf{0.189} \pm 0.015$	$\mathbf{0.123} \pm 0.009$	$\mathbf{0.339} \pm 0.026$
Brusselator	FM	0.217 ± 0.016	0.658 ± 0.042	0.238 ± 0.019	1.046 ± 0.057
	AFM (ours)	$\mathbf{0.125} \pm 0.011$	$\mathbf{0.031} \pm 0.004$	$\mathbf{0.109} \pm 0.008$	$\mathbf{0.023} \pm 0.003$
Van der Pol	FM	$\mathbf{0.143} \pm 0.012$	0.161 ± 0.014	$\mathbf{0.211} \pm 0.018$	0.510 ± 0.031
	AFM (ours)	0.225 ± 0.017	$\mathbf{0.051} \pm 0.005$	0.282 ± 0.021	$\mathbf{0.101} \pm 0.009$

Baselines. To evaluate the effectiveness of the autoregressive factorization in AFM, we compared our approach against a non-autoregressive flow-matching baseline, which directly models the joint distribution of future observations conditioned on past observations and covariates, that is, $p^1 = p(\mathbf{Y}_f \mid \mathbf{Y}_l, \mathbf{C})$. This approach serves as a representative non-autoregressive FM baseline; note that it differs from AFM in prior distribution and architecture in addition to factorization. Note that this non-autoregressive formulation closely resembles the method proposed by Kollovieh et al. (2025). Further details on this baseline are provided in Appendix A.

Evaluation metrics. To assess performance both within and beyond the forecast horizon used during training, we partitioned each trajectory into three segments: (1) an observation window (steps 1–75), which serves as the conditioning input to the model; (2) a within-training-horizon window (steps 76–150), which, for the *training* trajectories, is the forecast target used to fit the model; and (3) a beyond-training-horizon window (steps 151–200),

which the model is never trained to forecast. All reported metrics are computed on the 400 held-out *test* trajectories, no part of which is used for training. For these trajectories, both the within- and beyond-horizon windows are therefore genuinely out-of-sample; the two windows differ only in whether the forecast steps lie inside or outside the horizon length the model was trained on. Given an n-dimensional time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \ldots, \mathbf{y}_{\tau+f}\}$ and its predictive distribution or samples from our probabilistic model, we used the continuous ranked probability score (CRPS) (Gneiting and Raftery, 2007) for evaluating the performance of the model in uncertainty quantification and the normalized root mean square error (NRMSE) (Hyndman and Koehler, 2006) to assess prediction accuracy.

The continuous ranked probability score (CRPS) is a proper scoring rule commonly used to assess the quality of probabilistic forecasts. For a univariate random variable X with cumulative distribution function (CDF) F_X and an observed value x, the CRPS is defined as:

$$\mathrm{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} \big(F_X(y) - \mathbf{1}\{y \geq x\}\big)^2 \, \mathrm{d}y, \tag{9}$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. Intuitively, the CRPS evaluates how well the entire predictive distribution aligns with the observation x. Lower CRPS values indicate better-calibrated and more accurate predictive distributions. In the context of time series forecasting, we can compute the CRPS for each time step in the forecasting horizon and average the results: $\overline{\mathrm{CRPS}} = \frac{1}{f}\sum_{k=1}^{f} \mathrm{CRPS}\big(F_{X_{\tau+k}}, y_{\tau+k}\big)$, where $F_{X_{\tau+k}}$ is the predicted CDF (or an empirical CDF from samples) at forecast horizon k, and $y_{\tau+k}$ is the corresponding ground-truth observation. For the synthetic systems (Table 2) and the degradation benchmarks (Table 6 and Appendix B), we compute this score from S forecast samples sorted as $x_{(1)} \leq \cdots \leq x_{(S)}$ by evaluating the integrand at the samples and averaging with equal weights, $\widehat{\mathrm{CRPS}} = \frac{1}{S}\sum_{i=1}^{S} \big(\frac{i}{S} - \mathbf{1}\{x_{(i)} \geq y\}\big)^2$. This amounts to integrating with respect to the predictive distribution rather than over the real line: the score depends only on the rank of the observation among the samples, ranging from 1/12 (observation at the predictive median) to 1/3 (observation outside the range of the samples), and it does not penalise an over-dispersed predictive distribution. It therefore reflects where the observation falls within the predictive distribution rather than its sharpness, which is why we report it alongside NRMSE and, for the degradation benchmarks, prediction-interval coverage.

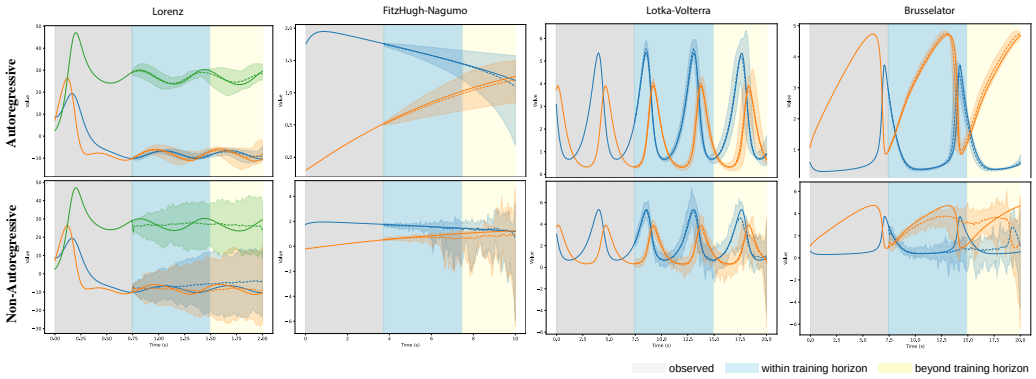


Figure 2: Example probabilistic forecasts from AFM (top row) and the non-autoregressive FM baseline (bottom row) for one held-out test trajectory per dynamical system. Solid lines: ground truth; dashed lines: predictive mean; shaded bands: 95% prediction intervals. Grey: observed window (model input, steps 1–75). Blue: within the training horizon (steps 76–150). Yellow: beyond the training horizon (steps 151–200). All trajectories shown belong to the held-out test set, so no part of them (in either the blue or the yellow window) was seen during training.

The real-world results in Table 4 instead use the standard quantile-based CRPS computed by GluonTS (Section 4.2).

The root mean square error (RMSE) is a standard metric for evaluating the accuracy of point forecasts. For a set of predictions $\{\hat{y}_{\tau+1}, \ldots, \hat{y}_{\tau+f}\}$ and corresponding ground-truth values $\{y_{\tau+1}, \ldots, y_{\tau+f}\}$, the RMSE is:

$$\mathrm{RMSE} = \sqrt{\frac{1}{f}\sum_{k=1}^{f}\left(\hat{y}_{\tau+k} - y_{\tau+k}\right)^2}. \tag{10}$$

To facilitate comparison across datasets of different magnitudes, we normalize by the sample standard deviation of the ground-truth series, giving $\mathrm{NRMSE} = \mathrm{RMSE}/\sigma_y$. A lower NRMSE indicates better predictive accuracy relative to the variability of the time series.

Results. Our results, summarized in Table 2, demonstrate that the autoregressive factorization employed in AFM significantly improves forecasting performance across multiple stochastic dynamical systems. Within the training horizon, the AR approach outperforms the FM baseline in terms of NRMSE across all five systems, with particularly dramatic improvements

observed in the Lorenz (0.017 vs 0.242) and Brusselator (0.031 vs 0.658) systems, representing reductions in error of 93% and 95%, respectively. The AR factorization also yields superior CRPS in four of the five systems, with only the Van der Pol oscillator showing better uncertainty quantification with the FM approach. More importantly, these improvements extend beyond the training horizon, where autoregressive factorization maintains its advantage in NRMSE across all systems, demonstrating its superior ability to forecast beyond the horizon it was trained on. This is particularly evident in the Brusselator system, where AR factorization reduces the beyond-horizon NRMSE by 98% (0.023 vs 1.046).

An exception is Van der Pol, where FM achieves lower CRPS than AFM despite AFM winning on NRMSE. Van der Pol with $\mu = 0.1$ is a nearly harmonic oscillator with smooth, low-entropy dynamics; in this regime the FM baseline's trajectory-level prior may better match the shape of the marginal predictive distribution, while AFM's step-wise sampling is more accurate in mean trajectory but yields slightly different marginal shapes due to autoregressive error propagation.

Figure 2 illustrates the forecasting results for AFM compared to FM by generating future trajectories for four dynamical systems conditioned on observed data from held-out test trajectories. AFM is shown to have much lower variance compared to the FM baseline, even allowing for accurate extrapolation of the dynamics beyond the forecast horizon it was trained on. These results confirm that modeling the temporal dependencies through autoregressive factorization enables AFM to better capture the underlying dynamics of complex stochastic systems, leading to more accurate predictions and improved uncertainty quantification, especially in challenging extrapolation scenarios.

4.2. Real-world datasets

Data. We evaluated AFM on the tasks of forecasting multiple univariate and multivariate timeseries with different frequencies (hourly and daily) from GluonTS (Alexandrov et al., 2020). Specifically, we used the following datasets: Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua and Graff, 2017), Traffic (Dua and Graff, 2017), and Wikipedia (Gasthaus et al., 2019). We used the versions preprocessed as in Salinas et al. (2019). Evaluation is performed on a held-out test set consisting of trajectories never seen during training; the model is trained exclusively on the training split

Table 3: Properties of the datasets used in the experiments: dimension n, domain Ω, frequency, total training timesteps T, and prediction length L.

Dataset	n	Ω	freq.	T	L
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	$(0, 1)$	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

Table 4: Test set CRPS comparison (lower is better). The reported metric is `mean_wQuantileLoss` computed by GluonTS `MultivariateEvaluator` on `SampleForecast` objects, over quantile grid $\{0.05, 0.10, \ldots, 0.95\}$, averaged over dimensions. This corresponds to marginal univariate CRPS averaged over dimensions. Mean and standard error metrics for AFM obtained by re-training and evaluating five times. Rows marked † were computed by us with `statsforecast` under exactly the same windows, evaluator and quantile grid as AFM, from quantile forecasts (Section 4.2); results for all other baselines are taken from Kollovieh et al. (2025) (for TSFlow, the conditional model as reported in the first arXiv version) and Rasul et al. (2021a). The naive forecast is reported only for the non-seasonal Exchange dataset and the multi-seasonal methods only for the hourly datasets with daily and weekly seasonality; "–" marks combinations that are not applicable. Best scores in bold and second best underlined.

Method	Electricity	Exchange	Solar	Traffic	Wikipedia
Statistical baselines					
Naive†	–	**0.007** ± 0.000	–	–	–
SN (single season)	0.069 ± 0.000	–	0.512 ± 0.000	0.221 ± 0.000	0.423 ± 0.000
AutoARIMA (single season)	0.344 ± 0.000	<u>0.008</u> ± 0.000	0.558 ± 0.003	0.486 ± 0.000	0.654 ± 0.000
AutoETS (single season)	0.055 ± 0.000	<u>0.008</u> ± 0.000	0.550 ± 0.000	0.492 ± 0.000	0.651 ± 0.000
MSTL-ETS† (24, 168)	0.058 ± 0.000	–	0.520 ± 0.000	0.248 ± 0.000	–
DHR-ARIMA† (24, 168)	0.066 ± 0.000	–	0.460 ± 0.000	0.202 ± 0.000	–
Deep learning baselines					
DeepAR	0.051 ± 0.000	0.013 ± 0.004	0.429 ± 0.055	0.103 ± 0.002	0.215 ± 0.003
WaveNet	0.058 ± 0.008	0.012 ± 0.001	0.360 ± 0.009	0.099 ± 0.002	**0.207** ± 0.003
CSDI	0.051 ± 0.000	0.013 ± 0.001	0.360 ± 0.006	0.152 ± 0.001	0.318 ± 0.012
SSSD	0.048 ± 0.001	0.010 ± 0.001	0.384 ± 0.023	0.097 ± 0.002	<u>0.209</u> ± 0.004
TimeGrad	0.048 ± 0.001	0.013 ± 0.002	0.347 ± 0.024	0.109 ± 0.002	0.311 ± 0.002
TSFlow	<u>0.045</u> ± 0.001	0.009 ± 0.001	<u>0.343</u> ± 0.002	**0.083** ± 0.000	0.227 ± 0.000
AFM (ours)	**0.042** ± 0.001	0.009 ± 0.001	**0.284** ± 0.002	<u>0.089</u> ± 0.000	0.243 ± 0.002

and all reported metrics are computed on this separate test partition. A summary of their properties is listed in Table 3.

Implementation details. We set ζ_ϕ as a three-layer bi-directional LSTM with 64 hidden units. We defined ν_θ via a residual neural network with five residual blocks implemented via 1-D convolutional layers with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2021). The architecture is similar to the network used by Rasul et al. (2021a) but with fewer residual blocks. We used Fourier positional embeddings to encode the flow step s into a 32-dimensional vector using a fixed set of frequencies. We used the same embedding technique to encode time-dependent covariates $\mathbf{c}$. We trained our models via stochastic gradient descent using an Adam optimizer with a learning rate of 0.001 and a batch size of 128. Depending on the dataset, training until convergence took from $40 - 100$ minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines. We compared AFM against multiple established baselines. Following Hewamalage et al. (2023), the statistical benchmarks are matched to the characteristics of each dataset. For Exchange, whose daily exchange rates exhibit no seasonality, the appropriate benchmark is the naive (random-walk) forecast, which repeats the last observed value, with Gaussian prediction intervals whose variance grows linearly with the horizon; random-walk forecasts are notoriously difficult to beat for exchange rates (Meese and Rogoff, 1983). The hourly datasets (Electricity, Solar and Traffic) exhibit both daily ($m_1 = 24$) and weekly ($m_2 = 168$) seasonal cycles, so for these we include two methods designed for multiple seasonal patterns (Hyndman et al., 2025): (i) MSTL-ETS, which removes both seasonal components by MSTL decomposition (Bandara et al., 2025), forecasts the seasonally adjusted series with AutoETS, and forecasts each seasonal component with a seasonal naive method; and (ii) dynamic harmonic regression (DHR-ARIMA), an ARIMA model with K_1 and K_2 Fourier pairs for the daily and weekly periods, respectively, as regressors, where the ARIMA orders are selected automatically and $(K_1, K_2) \in \{2, 4, 6\}^2$ was chosen per dataset by minimising the AICc summed over 20 randomly selected series, giving $(6, 4)$, $(6, 2)$ and $(6, 6)$ for Electricity, Solar and Traffic, respectively. For reference, we also report Seasonal Naive (SN), AutoARIMA and AutoETS (Hyndman et al., 2008) with a single seasonal period ($m = 24$ for the hourly datasets; SN uses $m = 30$ for the daily datasets), which cannot represent both seasonal cycles simultaneously. All statistical baselines computed by us were fitted per series with `statsforecast` (version 2.1.1) (Garza et al., 2022) at the same forecast origins and horizons (test windows) as AFM, using all obser-

vations before each forecast origin for the naive forecast and the most recent 1344 observations (eight weeks) for MSTL-ETS and DHR-ARIMA, and were evaluated with the same evaluator and quantile grid; predictive quantiles $0.05, 0.10, \ldots, 0.95$ were obtained from the models' Gaussian prediction intervals at levels $10\%, 20\%, \ldots, 90\%$, with the point forecast as the median, and forecasts were clipped to each dataset's domain Ω (Table 3). The deep learning baselines comprise DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016), the diffusion baselines CSDI (Tashiro et al., 2021), SSSD (Alcaraz and Strodthoff, 2023), and TimeGrad (Rasul et al., 2021a), and the flow matching baseline TSFlow (Kollovieh et al., 2025).

Table 5: Model efficiency comparison. Columns show number of parameters, inference time per forecast, and CRPS on Electricity and Solar (lower is better for all three). AFM achieves the best predictive accuracy with the smallest model size, while being approximately 4× slower than the non-autoregressive TSFlow but significantly faster than TimeGrad. The accuracy and speed columns are independent metrics; lower CRPS does not imply faster inference.

Method	Parameters	Inference Time	Electricity	Solar
TSFlow	4.1M	**45ms**	0.045 ± 0.001	0.343 ± 0.002
TimeGrad	2.8M	850ms	0.048 ± 0.001	0.347 ± 0.024
AFM (ours)	**0.85M**	180ms	$\mathbf{0.042} \pm 0.001$	$\mathbf{0.284} \pm 0.002$

Results. Table 4 presents the comparison of our proposed AFM model against various baselines on five real-world timeseries datasets. The results show that AFM is competitive with strong baselines, achieving the best CRPS on two of the five datasets (Electricity and Solar). On the Electricity dataset, AFM outperforms all baselines with a CRPS of 0.042, representing a 6.7% improvement over the second-best model, TSFlow (0.045), and a 12.5% improvement over the autoregressive diffusion baseline TimeGrad (0.048). On the Exchange dataset, the best CRPS is obtained by the naive (random-walk) forecast (0.007), followed by AutoARIMA and AutoETS (both 0.008) and by AFM and TSFlow (both 0.009 ± 0.001); AFM does not improve on the naive forecast here, and the classical methods differ from one another by at most 0.001. That simple benchmarks are hard to beat on this dataset is consistent with the well-known near-random-walk behaviour of exchange rates (Meese and Rogoff, 1983). Most notably, on the Solar dataset, AFM achieves a substantial improvement with a CRPS of 0.284, representing a

17.2% reduction in error compared to TSFlow (0.343) and an 18.2% improvement over TimeGrad (0.347). On the Traffic dataset, AFM (0.089 ± 0.000) falls behind TSFlow (0.083 ± 0.000); while the difference is small, it is consistent across runs. On the Wikipedia dataset, WaveNet (0.207) and SSSD (0.209) achieve the best performance, with AFM (0.243) behind both TSFlow (0.227) and those baselines. On the three hourly datasets, the multi-seasonal baselines improve on the single-season ones in most but not all cases. DHR-ARIMA (0.066, 0.460 and 0.202 on Electricity, Solar and Traffic) is the best statistical method on Solar and Traffic, whereas on Electricity the single-season AutoETS (0.055) remains best, ahead of MSTL-ETS (0.058) and DHR-ARIMA. MSTL-ETS improves on the single-season AutoARIMA and AutoETS on Solar (0.520) and Traffic (0.248), but not on the seasonal naive forecast. All statistical baselines remain well behind the deep probabilistic models on these datasets.

Efficiency. Table 5 presents the computational efficiency analysis. AFM achieves the best predictive performance with the smallest model size (0.85M parameters compared to 4.1M for TSFlow and 2.8M for TimeGrad). In terms of inference time, AFM requires 180ms per forecast, which is 4× slower than the non-autoregressive TSFlow (45ms) but significantly faster than the autoregressive diffusion baseline TimeGrad (850ms). We note that AFM and TSFlow use different architectures and implementations, so this comparison should be interpreted as a rough order-of-magnitude characterization rather than a definitive efficiency benchmark. The slower inference of AFM relative to TSFlow reflects the sequential nature of autoregressive sampling and represents a trade-off against the extrapolation and multivariate scaling benefits. These results demonstrate that the autoregressive flow-matching approach employed in AFM delivers competitive performance across diverse real-world timeseries forecasting tasks, with favorable trade-offs between model size, inference speed, and predictive accuracy.

Additional physical-degradation benchmarks.. To broaden the evaluation beyond standard forecasting corpora, we additionally assess AFM on two physical-degradation datasets widely used in the prognostics literature: the NASA Li-ion Battery Aging dataset (capacity-fade forecasting under a leave-one-battery-out protocol) and the NASA C-MAPSS turbofan dataset (multivariate sensor-trajectory forecasting). We report a three-seed comparison of the autoregressive (AFM) and non-autoregressive (FM) factorizations, together

Dataset	Model	CRPS	CRPS-Sum	NRMSE	RMSE	cov_{50}	cov_{90}	corr-dist
Battery	AFM	0.302±0.010	0.263±0.015	0.621±0.080	0.078±0.020	0.059±0.019	0.179±0.059	0.560±0.114
	FM	0.230±0.008	0.218±0.007	0.498±0.007	0.034±0.002	0.249±0.021	0.533±0.050	1.457±0.913
Turbofan	AFM	0.180±0.004	0.181±0.003	0.731±0.045	0.349±0.024	0.439±0.018	0.821±0.017	2.020±0.739
	FM	0.179±0.003	0.193±0.006	0.685±0.008	0.332±0.002	0.435±0.013	0.842±0.013	2.410±0.258

Table 6: AFM-vs-FM comparison on the two real-world degradation benchmarks (mean±std). "CRPS-Sum" is the CRPS of the channel-summed forecast, an aggregate multivariate score; "corr-dist" is the Frobenius distance between the forecast and ground-truth cross-channel correlation matrices (lower is better for both). Per-horizon calibration/CRPS diagnostics, the cross-sensor correlation maps, and example probabilistic forecasts are provided in Appendix B.

with the calibration (prediction-interval coverage), horizon-resolved CRPS, and cross-channel correlation-structure diagnostics summarized in Table 6; the per-horizon diagnostics and forecast visualizations are provided in Appendix B. On the turbofan benchmark, AFM attains lower CRPS than FM over the near-to-mid forecast horizon and reproduces the empirical cross-sensor correlation structure more closely, whereas the whole-window FM is preferable at very long horizons. On the smoother battery capacity-fade signal, where per-step uncertainty is low, the non-autoregressive FM is the stronger and better-calibrated forecaster and AFM tends to be over-confident at long range. These results reinforce our central, measured claim: the autoregressive factorization is most beneficial for stochastic, multi-modal dynamics and near-to-mid horizons, while offering little advantage on smooth, near-deterministic decay where a global non-autoregressive prior suffices.

5. Discussion

In this work, we introduced AFM, a probabilistic forecasting model that combines autoregressive modeling with flow matching to generate accurate and well-calibrated predictions for multivariate timeseries data. By factorizing the forecasting problem into a sequence of conditional distributions, AFM effectively balances the sequential dependencies in temporal data with the expressiveness of flow-based generative modeling. Our empirical results across simulated dynamical systems and real-world datasets demonstrate that AFM achieves competitive performance against state-of-the-art baselines in challenging forecasting benchmarks. The autoregressive decomposition leads to improved CRPS and NRMSE in most evaluated settings compared to the non-autoregressive FM baseline, suggesting better calibration of marginal

predictive distributions. Furthermore, AFM exhibits robust generalization to out-of-distribution scenarios, particularly in extrapolation tasks where it achieves 93–98% error reduction compared to the non-autoregressive FM baseline on synthetic systems (e.g., Brusselator: NRMSE 0.023 vs 1.046). These improvements are observed on stochastic dynamical systems where the ground truth model is known; whether similar gains transfer to real-world settings where data is not generated by a low-order dynamical system remains an open question.

AFM has limitations that warrant discussion. First, the sequential nature of autoregressive sampling introduces computational overhead during inference compared to fully parallelizable non-autoregressive models. Our experiments show that AFM is $4\times$ slower than non-autoregressive flow matching approaches (180ms vs 45ms per forecast on real-world datasets), which may limit its applicability in real-time forecasting systems requiring latency below 50ms. However, AFM remains significantly faster than autoregressive diffusion baselines like TimeGrad (850ms) while achieving better predictive performance with smaller model sizes (0.85M vs 2.8M–4.1M parameters). A second limitation is calibration at long horizons: because AFM is trained on single-step conditionals with teacher forcing but rolls out on its own predictions at inference, accumulated error is not reflected in its predictive intervals, so the model can become over-confident far into the horizon (most visibly on smooth, near-deterministic signals such as battery capacity fade). Mitigating this exposure bias via multi-step or scheduled-sampling training is a promising direction for future work. Non-autoregressive methods are preferable when: (1) real-time inference is critical with strict latency requirements ($<$50ms), (2) forecast horizons are very short (1–5 steps) where the benefits of autoregressive modeling are minimal, and (3) training data fully covers the target prediction horizon and extrapolation beyond the training distribution is not required. In contrast, AFM excels in applications where accuracy and well-calibrated uncertainty estimates are prioritized over inference speed, such as climate modeling, demand planning, and scientific forecasting tasks that involve extrapolation. Additionally, our current implementation applies flow matching directly in the data space, which may be suboptimal for high-dimensional partially observed data.

Overall, AFM demonstrates the potential of autoregressive factorization within the flow matching framework for probabilistic timeseries forecasting, achieving competitive accuracy with a compact model. Future work could explore latent-space formulations, extension to irregularly sampled data,

adaptive window sizes, and broader applications in scientific modeling and decision-making under uncertainty.

References

Albergo, M.S., Boffi, N.M., Vanden-Eijnden, E., 2023. Stochastic interpolants: A unifying framework for flows and diffusions. arXiv preprint arXiv:2303.08797 .

Alcaraz, J.M.L., Strodthoff, N., 2023. Diffusion-based time series imputation and forecasting with structured state space models. Transactions on Machine Learning Research .

Alexandrov, A., Benidis, K., Bohlke-Schneider, M., Flunkert, V., Gasthaus, J., Januschowski, T., Maddix, D.C., Rangapuram, S., Salinas, D., Schulz, J., et al., 2020. GluonTS: Probabilistic and neural time series modeling in python. Journal of Machine Learning Research 21, 1–6.

Bandara, K., Hyndman, R.J., Bergmeir, C., 2025. MSTL: A seasonal-trend decomposition algorithm for time series with multiple seasonal patterns. International Journal of Operational Research 52, 79–98.

Box, G.E.P., Jenkins, G.M., Reinsel, G.C., 1994. Time Series Analysis: Forecasting and Control. 3 ed., Prentice Hall, Englewood Cliffs, N.J.

Chen, R.T.Q., Rubanova, Y., Bettencourt, J., Duvenaud, D.K., 2018. Neural ordinary differential equations. Advances in Neural Information Processing Systems 31.

Dama, F., Sinoquet, C., 2021. Time series analysis and modeling to forecast: a survey. arXiv preprint arXiv:2104.00164 .

Desai, A., Freeman, C., Wang, Z., Beaver, I., 2021. TimeVAE: A variational auto-encoder for multivariate time series generation. arXiv preprint arXiv:2111.08095 .

Dua, D., Graff, C., 2017. UCI machine learning repository. University of California, Irvine, School of Information and Computer Sciences. `http://archive.ics.uci.edu/ml`.

Esser, P., Kulal, S., Blattmann, A., Entezari, R., Müller, J., Saini, H., Levi, Y., Lorenz, D., Sauer, A., Boesel, F., et al., 2024. Scaling rectified flow transformers for high-resolution image synthesis, in: Forty-first International Conference on Machine Learning.

FitzHugh, R., 1961. Impulses and physiological states in theoretical models of nerve membrane. Biophysical Journal 1, 445–466. doi:10.1016/S0006-3495(61)86902-6.

Garza, A., Mergenthaler Canseco, M., Challú, C., Olivares, K.G., 2022. StatsForecast: Lightning fast forecasting with statistical and econometric models, in: PyCon Salt Lake City, Utah, US 2022. https://github.com/Nixtla/statsforecast.

Gasthaus, J., Benidis, K., Wang, Y., Rangapuram, S.S., Salinas, D., Flunkert, V., Januschowski, T., 2019. Probabilistic forecasting with spline quantile function RNNs, in: The 22nd International Conference on Artificial Intelligence and Statistics, PMLR. pp. 1901–1910.

Gneiting, T., Raftery, A.E., 2007. Strictly proper scoring rules, prediction, and estimation. Journal of the American Statistical Association 102, 359–378.

Grathwohl, W., Chen, R.T.Q., Bettencourt, J., Sutskever, I., Duvenaud, D., 2019. FFJORD: Free-Form continuous dynamics for scalable reversible generative models, in: International Conference on Learning Representations.

Hewamalage, H., Ackermann, K., Bergmeir, C., 2023. Forecast evaluation for data scientists: Common pitfalls and best practices. Data Mining and Knowledge Discovery 37, 788–832.

Ho, J., Jain, A., Abbeel, P., 2020. Denoising diffusion probabilistic models. Advances in Neural Information Processing Systems 33, 6840–6851.

Hu, Y., Wang, X., Wu, L., Zhang, H., Li, S.Z., Wang, S., Chen, T., 2024. FM-TS: Flow matching for time series generation. arXiv preprint arXiv:2411.07506 .

Hyndman, R., Koehler, A.B., Ord, J.K., Snyder, R.D., 2008. Forecasting with Exponential Smoothing: The State Space Approach. Springer Science & Business Media.

Hyndman, R.J., Athanasopoulos, G., Garza, A., Challu, C., Mergenthaler, M., Olivares, K.G., 2025. Forecasting: Principles and Practice, the Pythonic Way. OTexts, Melbourne, Australia. https://otexts.com/fpppy/.

Hyndman, R.J., Koehler, A.B., 2006. Another look at measures of forecast accuracy. International Journal of Forecasting 22, 679–688.

Karl, M., Soelch, M., Bayer, J., Van der Smagt, P., 2016. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. arXiv preprint arXiv:1605.06432 .

Kollovieh, M., Ansari, A.F., Bohlke-Schneider, M., Zschiegner, J., Wang, H., Wang, Y.B., 2023. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. Advances in Neural Information Processing Systems 36, 28341–28364.

Kollovieh, M., Lienen, M., Lüdke, D., Schwinn, L., Günnemann, S., 2025. Flow matching with gaussian process priors for probabilistic time series forecasting, in: The Thirteenth International Conference on Learning Representations.

Kong, Z., Ping, W., Huang, J., Zhao, K., Catanzaro, B., 2021. DiffWave: A versatile diffusion model for audio synthesis, in: International Conference on Learning Representations.

Lai, G., Chang, W.C., Yang, Y., Liu, H., 2018. Modeling long- and short-term temporal patterns with deep neural networks, in: The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, pp. 95–104.

Lefever, R., Prigogine, I., 1968. Symmetry breaking instabilities in dissipative systems. II. The Journal of Chemical Physics 48, 1695–1700. doi:10.1063/1.1668896.

Lim, B., Zohren, S., 2021. Time-series forecasting with deep learning: A survey. Philosophical Transactions of the Royal Society A 379, 20200209.

Lipman, Y., Chen, R.T.Q., Ben-Hamu, H., Nickel, M., Le, M., 2023. Flow matching for generative modeling, in: The Eleventh International Conference on Learning Representations.

Liu, X., Gong, C., Liu, Q., 2023. Flow straight and fast: Learning to generate and transfer data with rectified flow, in: The Eleventh International Conference on Learning Representations.

Liu, Y., Cheng, M., Wang, D., Tao, X., Liu, Q., 2026. CoGenCast: A coupled autoregressive–flow generative framework for time series forecasting. arXiv preprint arXiv:2602.03564 .

Lorenz, E.N., 1963. Deterministic nonperiodic flow. Journal of the Atmospheric Sciences 20, 130–141.

Lotka, A.J., 1925. Elements of Physical Biology. Williams & Wilkins.

Makridakis, S., Spiliotis, E., Assimakopoulos, V., 2020. The M4 competition: 100,000 time series and 61 forecasting methods. International Journal of Forecasting 36, 54–74.

Meese, R.A., Rogoff, K., 1983. Empirical exchange rate models of the seventies: Do they fit out of sample? Journal of International Economics 14, 3–24.

Meijer, C., Chen, L.Y., 2024. The rise of diffusion models in time-series forecasting. arXiv preprint arXiv:2401.03006 .

Petropoulos, F., Apiletti, D., Assimakopoulos, V., et al., 2022. Forecasting: Theory and practice. International Journal of Forecasting 38, 705–871.

van der Pol, B., 1926. On 'relaxation-oscillations'. The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science 2, 978–992. doi:10.1080/14786442608564127.

Polyak, A., Zohar, A., Brown, A., Tjandra, A., Sinha, A., Lee, A., Vyas, A., Shi, B., Ma, C.Y., Chuang, C.Y., et al., 2024. MovieGen: A cast of media foundation models. arXiv preprint arXiv:2410.13720 .

Rasul, K., Seward, C., Schuster, I., Vollgraf, R., 2021a. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting, in: International Conference on Machine Learning, PMLR. pp. 8857–8868.

Rasul, K., Sheikh, A.S., Schuster, I., Bergmann, U., Vollgraf, R., 2021b. Multivariate probabilistic time series forecasting via conditioned normalizing flows, in: International Conference on Learning Representations.

Richardson, R.R., Osborne, M.A., Howey, D.A., 2017. Gaussian process regression for forecasting battery state of health. Journal of Power Sources 357, 209–219.

Salinas, D., Bohlke-Schneider, M., Callot, L., Medico, R., Gasthaus, J., 2019. High-dimensional multivariate forecasting with low-rank gaussian copula processes. Advances in Neural Information Processing Systems 32.

Salinas, D., Flunkert, V., Gasthaus, J., Januschowski, T., 2020. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. International Journal of Forecasting 36, 1181–1191.

Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., Ganguli, S., 2015. Deep unsupervised learning using nonequilibrium thermodynamics, in: International Conference on Machine Learning, PMLR. pp. 2256–2265.

Song, Y., Sohl-Dickstein, J., Kingma, D.P., Kumar, A., Ermon, S., Poole, B., 2021. Score-based generative modeling through stochastic differential equations, in: International Conference on Learning Representations.

Tamir, E., Laabid, N., Heinonen, M., Garg, V., Solin, A., 2024. Conditional flow matching for time series modelling, in: ICML Workshop: Structured Probabilistic Inference and Generative Modeling.

Tashiro, Y., Song, J., Song, Y., Ermon, S., 2021. CSDI: Conditional score-based diffusion models for probabilistic time series imputation. Advances in Neural Information Processing Systems 34, 24804–24816.

Tong, A., Fatras, K., Malkin, N., Huguet, G., Zhang, Y., Rector-Brooks, J., Wolf, G., Bengio, Y., 2024. Improving and generalizing flow-based generative models with minibatch optimal transport. Transactions on Machine Learning Research .

Van Den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A., Kavukcuoglu, K., et al., 2016. WaveNet: A generative model for raw audio. arXiv preprint arXiv:1609.03499 .

Williams, R.J., Zipser, D., 1989. A learning algorithm for continually running fully recurrent neural networks. Neural Computation 1, 270–280.

Winkler, R.L., Munoz, J., Cervera, J.L., Bernardo, J.M., Blattenberger, G., Kadane, J.B., Lindley, D.V., Murphy, A.H., Oliver, R.M., Ríos-Insua, D., 1996. Scoring rules and the evaluation of probabilities. Test 5, 1–60.

Ye, J., Zhang, W., Yi, K., Yu, Y., Li, Z., Li, J., Tsung, F., 2024. A survey of time series foundation models: Generalizing time series representation with large language model. arXiv preprint arXiv:2405.02358 .

Appendix A. Standard FM baseline

Problem Setting. We consider the same forecasting setup as in Section 3.2 where given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, our goal remains to model the conditional distribution $p(\mathbf{Y}_f \mid \mathbf{Y}_l, \mathbf{C})$. However, unlike the autoregressive factorization, we directly model the joint distribution of the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ without temporal factorization:

$$p(\mathbf{Y}_f \mid \mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \ldots, \mathbf{y}_{\tau+f} \mid \mathbf{Y}_l, \mathbf{C})\,. \tag{A.1}$$

This formulation preserves temporal correlations across all future time steps but requires learning a high-dimensional distribution.

Training. We construct a probability path $(p^s(\mathbf{Y}_f))^{0 \le s \le 1}$ that transports samples from a prior distribution $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \mathbf{\Sigma})$ to the target distribution $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f \mid \mathbf{Y}_l, \mathbf{C})$, where $\mathbf{\Sigma}$ is a block-diagonal covariance matrix. The prior can be interpreted as independent Brownian motion processes per dimension, where Brownian motion is defined as a stochastic process $W\colon [\tau+1, \tau+f] \to \mathbb{R}^n$.

We define the conditional probability path using a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f \mid \mathbf{z}) = \mathcal{N}\left(\mathbf{m}_s, \sigma^2 s(1-s)\mathbf{\Sigma}\right) \tag{A.2}$$

where $\mathbf{m}_s = (1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1$ and $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field becomes:

$$\mu(\mathbf{Y}_f, s \mid \mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\mathbf{\Sigma}^{-1}(\mathbf{Y}_f - \mathbf{m}_s) \tag{A.3}$$

When $\sigma^2 \to 0$, this reduces to the straight path velocity $\mu(\mathbf{Y}_f, s \mid \mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. We learn a neural velocity field ν_θ that operates on the entire future trajectory. The context encoding remains similar to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \tag{A.4}$$

where ζ_ϕ now processes both past and future covariates. The training objective becomes:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}\left\|\mu(\mathbf{Y}_f^s, s \mid \mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\right\|^2 \tag{A.5}$$

Algorithm 3 FM training.

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ

while not converged **do**
  Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \mathbf{\Sigma})$
  Sample $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
  Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
  Sample $s \sim \mathcal{U}(0, 1)$
  Sample $\mathbf{Y}_f^s \sim \mathcal{N}\left(\mathbf{m}_s, \sigma^2 s(1-s)\mathbf{\Sigma}\right)$ with $\mathbf{m}_s = (1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1$
  Set target $\mu(\mathbf{Y}_f^s, s \mid \mathbf{z})$
  Set $\mathcal{L} = \|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2_{\mathbf{\Sigma}^{-1}}$
  Update θ, ϕ to minimize the loss
end while

Algorithm 4 FM inference.

Require: $\mathbf{Y}_l$, $\mathbf{C}$, networks ν_θ, ζ_ϕ

Encode context $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \mathbf{\Sigma})$
Solve $\frac{\mathrm{d}\psi(\mathbf{Y}_f, s)}{\mathrm{d}s} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
Return $\psi(\mathbf{Y}_f, 1)$

where the expectation is taken over $\mathbf{z} \sim \pi_{0,1}$, $s \sim \mathcal{U}(0, 1)$ and $\mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f \mid \mathbf{z})$. The vector field ν_θ is implemented as a sequential neural network (e.g., Transformer or RNN) that processes the entire trajectory. Sampling requires solving the trajectory-level ODE:

$$\frac{\mathrm{d}\psi(\mathbf{Y}_f, s)}{\mathrm{d}s} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s) \tag{A.6}$$

using numerical solvers subject to $\psi(\mathbf{Y}_f, 0) \sim p^0$. This produces joint samples from $p(\mathbf{Y}_f \mid \mathbf{Y}_l, \mathbf{C})$ without autoregressive decomposition.

Algorithms 3 and 4 describe training and inference procedures for the standard FM model, respectively.

Implementation Details. We used this baseline to conduct experiments on the task of forecasting multiple dynamical systems. For this setup we set ν_θ as a 4-layer bi-directional LSTM with 128 hidden units. We kept the same training parameters as in Section 4.1. Training until convergence took $\sim$2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant. The non-autoregressive approach differs fundamentally from its autoregressive counterpart introduced in AFM. First, while the autoregressive method factorizes the joint distribution via a Markovian decomposition across time steps, the non-autoregressive baseline directly models the full future trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable. This preserves cross-temporal dependencies at the expense of learning a more complex distribution over $\mathbb{R}^{n \times f}$. Second, computational characteristics diverge significantly: the non-autoregressive version processes entire trajectories through sequential neural networks for the velocity fields enabling parallel generation of all future timepoints at the cost of higher memory requirements. Finally, their uncertainty propagation mechanisms contrast sharply: the autoregressive approach explicitly models how prediction errors accumulate through the chain of conditional distributions, while the non-autoregressive method captures joint uncertainty over all timesteps through trajectory-level sampling but lacks explicit mechanisms to model error accumulation dynamics. These differences create complementary trade-offs as the autoregressive variant offers interpretable uncertainty quantification and efficient window-based computation but assumes limited Markovian dependencies. In contrast, the non-autoregressive baseline preserves full temporal correlations at higher computational cost with less explicit uncertainty dynamics.

Appendix B. Degradation benchmarks

This appendix provides the supporting material for the two real-world physical-degradation benchmarks introduced in the main text (Section 4.2): the NASA Li-ion battery (per-cycle discharge-capacity fade, leave-one-battery-out; 66-cycle context and horizon) and the C-MAPSS turbofan (14-sensor multivariate forecasting; 20-cycle context, 108-cycle horizon). The aggregate three-seed AFM-vs-FM comparison is reported in the main text (Table 6); here we give the per-horizon calibration and CRPS curves, the cross-sensor correlation structure, example probabilistic forecasts, and the supporting accuracy and diagnostic tables.

Method	capacity RMSE (Ah)	norm. SOH RMSE
AFM (ours)	0.078±0.020	0.039
FM (ours)	0.034±0.002	0.017
GPR (Richardson et al., 2017)	–	≈0.06–0.08

Table B.7: Battery capacity-fade forecasting accuracy. AFM/FM are our three-seed means; the normalized-SOH column divides the capacity RMSE by the ∼2 Ah rated capacity. The matching-protocol published baseline is the Gaussian-process forecaster of Richardson et al. (2017).

Metric	AFM	FM	nominal/best
coverage@50%	0.439±0.018	0.435±0.013	0.50
coverage@90%	0.821±0.017	0.842±0.013	0.90
cross-sensor corr-dist	2.020±0.739	2.410±0.258	0

Table B.8: C-MAPSS turbofan calibration and dependence diagnostics (three-seed mean±std).

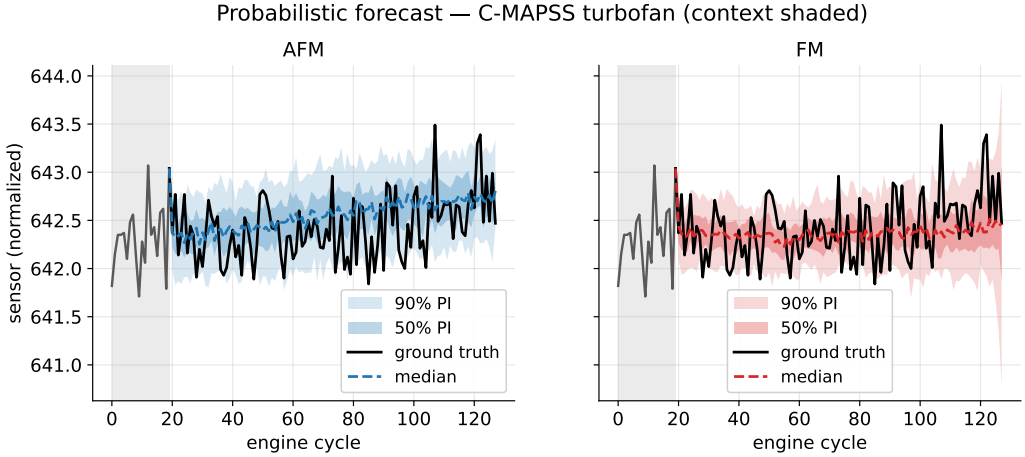


Figure B.3: C-MAPSS turbofan: probabilistic sensor-trajectory forecast for one held-out engine, AFM (left) vs. FM (right). Shaded context window, ensemble median, 50%/90% prediction intervals, and ground truth.

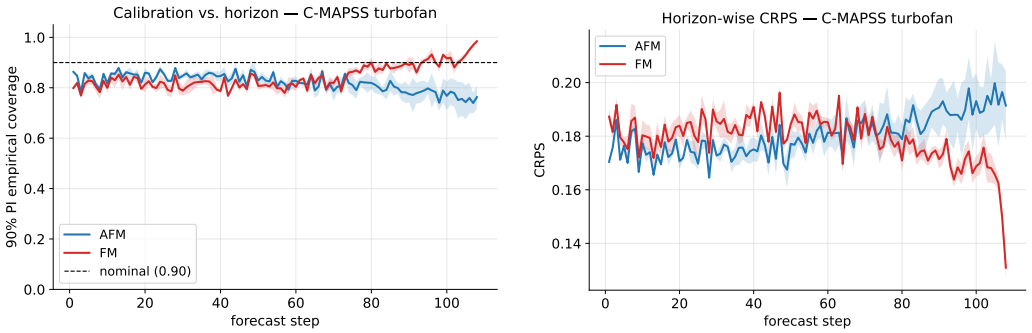


Figure B.4: C-MAPSS turbofan, three-seed mean±band. Left: 90% prediction-interval coverage vs. forecast horizon (nominal 0.9 dashed). Right: horizon-resolved CRPS. AFM is better calibrated and lower-CRPS over the near-to-mid horizon; FM overtakes at long range as AFM's rollout accumulates error. The sharp decrease in FM CRPS over the last few forecast steps does not reflect improved forecasts. FM, trained on 40-step windows, generates the 108-step horizon in one pass, and its predictive spread grows sharply over the final steps (at step 108 the 90% interval is 3.6× its average width over steps 1–100, and coverage rises to 0.985), while its point error does not decrease. The observations therefore fall closer to the centre of an over-dispersed predictive distribution, which the sample-rank CRPS of Section 4.1 does not penalise. Excluding the last four steps changes FM's aggregate CRPS in Table 6 by 0.001.

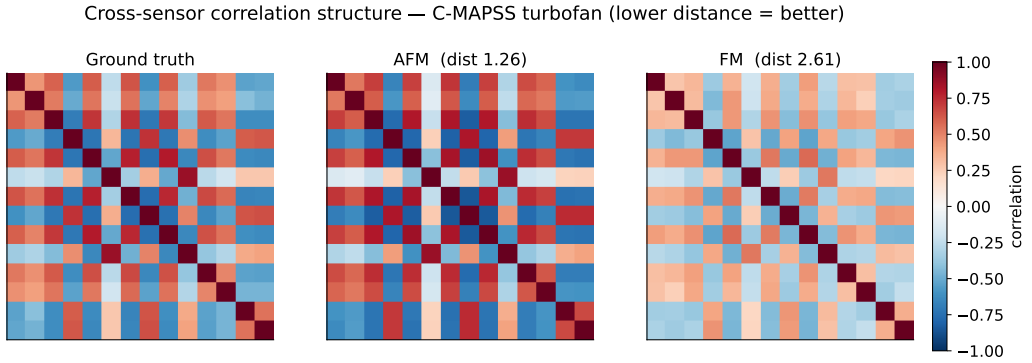


Figure B.5: C-MAPSS turbofan cross-sensor correlation matrices: ground truth vs. AFM and FM forecast samples (a smaller Frobenius distance to the ground-truth matrix indicates better-captured multivariate dependence).

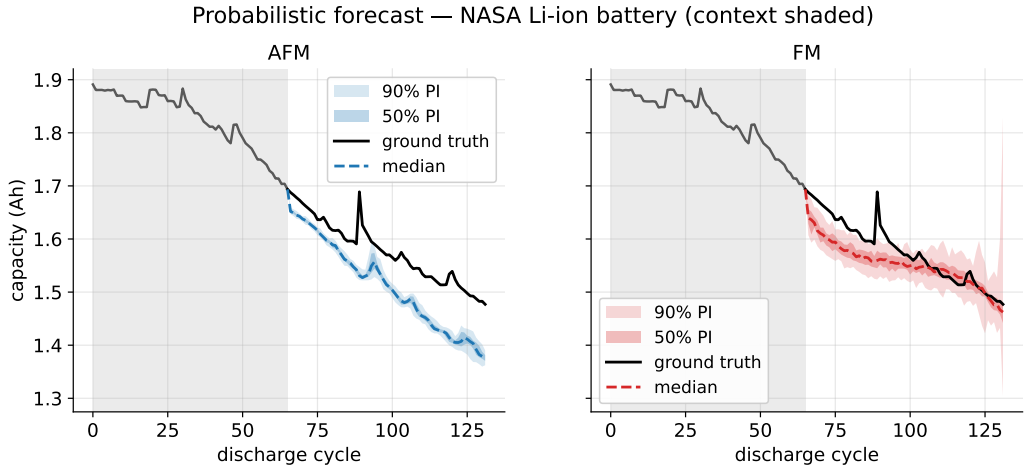


Figure B.6: NASA Li-ion battery (held-out cell): capacity-fade forecast, AFM (left) vs. FM (right). FM tracks the smooth decay more accurately and with better-calibrated intervals; AFM drifts low and is over-confident at long range.

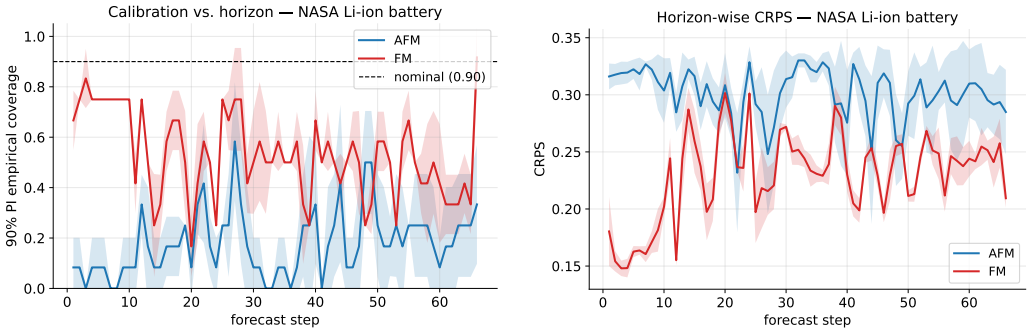


Figure B.7: NASA Li-ion battery, three-seed mean±band: 90% prediction-interval coverage (left) and horizon-resolved CRPS (right). Both methods under-cover on this smooth signal, but AFM is markedly more over-confident.